RUTA DE CRECIMIENTO PRÁCTICA
FLUTTER - DART

Dolph Brandon Hincapié

Senior Flutter Developer - Technical Officer SomosAleteo

2024

1 INTRODUCCIÓN

En este informe, exploraremos una ruta de crecimiento para aquellos que deseen dominar Flutter y aprovechar todo su potencial. Desde los conceptos básicos hasta técnicas avanzadas, analizaremos los pasos clave para dominar esta plataforma y construir aplicaciones móviles de alta calidad que satisfagan las demandas del mercado actual. A lo largo del camino, examinaremos las mejores prácticas, herramientas y recursos que facilitarán el proceso de aprendizaje y desarrollo en Flutter.

1.1 Propósito del instructivo

Proporcionar un camino estructurado y progresivo para que desarrolladores de todos los niveles adquieran habilidades sólidas en el desarrollo de aplicaciones multiplataforma utilizando Flutter.

Esta ruta tiene como objetivo ayudar a los desarrolladores a:

1. Aprender los Fundamentos: Comenzar con los conceptos básicos de Flutter, comprensión de la arquitectura y la sintaxis del lenguaje Dart.

2. Dominar los Componentes: Familiarizarse con los widgets y componentes de Flutter para construir interfaces de usuario atractivas y funcionales.

3. Explorar la Navegación y la Gestión de Estados: Aprender cómo manejar la navegación entre pantallas y gestionar el estado de la aplicación de manera efectiva para una experiencia de usuario fluida.

4. Integrar Funcionalidades Avanzadas: Adquirir conocimientos sobre cómo integrar características avanzadas como la autenticación de usuarios, el acceso a bases de datos, la conectividad a API, entre otros.

5. Optimizar el Rendimiento: Aprender técnicas para optimizar el rendimiento de las aplicaciones Flutter, incluyendo la gestión de recursos, la optimización del tiempo de carga y la mejora de la fluidez de la interfaz de usuario.

6. Practicar y Probar: Practicar el desarrollo de aplicaciones reales y realizar pruebas para garantizar la calidad y fiabilidad de las aplicaciones construidas con Flutter.

7. Actualizar Constantemente: Mantenerse al día con las últimas actualizaciones y tendencias en el ecosistema de Flutter para seguir mejorando y expandiendo las habilidades de desarrollo.

2 FASES RUTA DE CRECIMIENTO

2.1 FASE 1 - Aplicación básica en Flutter.

Objetivo

El objetivo de esta prueba técnica es evaluar tus habilidades para desarrollar una aplicación básica en Flutter. La aplicación consistirá en una pantalla principal con un listado de tarjetas que muestran información, una pantalla de detalles que muestra más información sobre un elemento seleccionado y una pantalla de formulario para agregar nuevos elementos al listado.

Requerimientos

1. Pantalla Principal (HomeScreen):
    - Mostrar un listado de tarjetas, cada una representando un elemento de datos.
    - Al hacer clic en una tarjeta, navegar a la pantalla de detalles correspondiente.
    - Incluir un botón que navegue a la pantalla de formulario.
2. Pantalla de Detalles (DetailsScreen):
    - Mostrar más información sobre el elemento seleccionado de la pantalla principal.
    - Incluir unos botones para editar y eliminar dicho elemento.
3. Pantalla de Formulario (FormScreen):
    - Permitir al usuario ingresar información para agregar un nuevo elemento a la pantalla principal o editar uno ya creado.
    - Incluir un botón para agregar el nuevo elemento.

Entregables

1. Código fuente de la aplicación en Flutter en Github.
2. Documentación breve en un readme dentro del código fuente, explicando el diseño de la aplicación y cualquier consideración importante en el código.

Evaluación

Tu prueba será evaluada en base a los siguientes criterios:

- Funcionalidad: La aplicación debe cumplir con todos los requerimientos mencionados.
- Estructura del código: El código debe estar bien organizado y seguir las mejores prácticas de Flutter.
- Diseño de la interfaz de usuario: La interfaz de usuario debe ser clara, intuitiva y estéticamente agradable.
- Manejo de la navegación: La navegación entre pantallas debe ser fluida y sin errores.
- Manejo de datos: La aplicación debe manejar correctamente los datos, incluyendo la adición y eliminación de elementos.

Instrucciones

1. Desarrolla la aplicación en Flutter siguiendo los requerimientos mencionados.
2. Documenta de manera externa cualquier decisión de diseño o implementación que consideres importante.
3. Una vez completada la prueba, envía el código fuente, la documentación y las capturas de pantalla o grabación de video.
4. Acorde al resultado generado por el evaluador se puede proseguir a la Fase 2 o en caso contrario revisar la ANEXO 1 al final.

2.2 FASE 2 - Lectura de Datos API.

Objetivo

El objetivo de esta prueba técnica es evaluar tus habilidades en Dart, incluida la lectura de una API externa, la creación de modelos de datos y la manipulación de estos datos para imprimirlos en la consola.

Requerimientos

1. Consumo de la API Fake Store:
    - Utiliza Dart para realizar solicitudes a al menos tres endpoints diferentes de la API Fake Store para obtener datos ficticios de una tienda en línea.
2. Creación de Modelos de Datos:
    - Crea modelos de datos en Dart para representar la información obtenida de la API.
    - Debes incluir al menos un modelo para representar los datos principales obtenidos de cada endpoint, manejar inmutabilidad en los modelos.
3. Control de Errores con Either:
    - Implementa un control de errores utilizando el tipo Either de la biblioteca dartz.
    - Maneja los posibles errores que puedan ocurrir al realizar las solicitudes a la API.
4. Impresión en Consola:
    - Utiliza Dart para procesar los datos obtenidos de la API y mostrarlos de manera legible en la consola.
    - Asegúrate de incluir información relevante de los modelos de datos, como el título, descripción, precio, etc.

Entregables

1. Código fuente de la aplicación en Dart en github.
2. Documentación explicando el diseño de los modelos de datos, cómo se realizó la solicitud y procesamiento de la API e implementación del control de errores con Either.

Evaluación

Tu prueba será evaluada en base a los siguientes criterios:

- Correctitud de los datos: Los datos obtenidos de la API deben ser precisos y estar representados correctamente en los modelos de datos.
- Estructura del código: El código debe estar bien organizado, seguir las convenciones de Dart y ser fácil de entender.
- Manejo de la API: Las solicitudes a la API deben realizarse de manera eficiente y sin errores, manejando los errores de forma adecuada con Either.
- Diseño de los modelos de datos: Los modelos de datos deben estar diseñados de manera adecuada para representar la información de la API de manera clara y coherente.
- Presentación en consola: Los datos impresos en la consola deben ser legibles y mostrar información relevante.

Instrucciones

1. Utiliza Dart para consumir al menos tres endpoints diferentes de la API Fake Store y crear modelos de datos correspondientes.
2. Implementa el control de errores utilizando Either para manejar posibles errores en las solicitudes a la API.
3. Procesa los datos obtenidos de la API y muéstralos en la consola de manera legible.
4. Documenta cualquier decisión de diseño o implementación que consideres importante.

2.3 FASE 3 - Crear Paquete y Publicar en Github.

Objetivo

El objetivo de esta prueba técnica es evaluar tus habilidades para crear un paquete para Flutter que utilice Dart para interactuar con la Fake Store API. Además, se espera que incluyas un ejemplo en Flutter que muestre la lectura de la API utilizando el paquete creado.

Requerimientos

1. Creación del Paquete para Flutter:
    - Crea un paquete de Flutter que utilice Dart para interactuar con la Fake Store API.
    - El paquete debe proporcionar métodos para realizar solicitudes a la API y procesar los datos obtenidos.
2. Example en Flutter:
    - Incluye un ejemplo en Flutter que demuestre el uso del paquete creado.
    - El ejemplo debe mostrar la lectura de la Fake Store API y la visualización de los datos obtenidos en una pantalla de la aplicación Flutter.

3. Publicación en GitHub:
    - Publica el paquete para Flutter en GitHub.
    - Asegúrate de incluir instrucciones claras sobre cómo usar el paquete y cómo ejecutar el ejemplo en Flutter.

Recursos

1. Como crear un paquete en Flutter: VIDEO EXPLICATIVO

Entregables

1. Código fuente del paquete para Flutter.
2. Código fuente del ejemplo en Flutter que utiliza el paquete.
3. Documentación detallada que explique cómo usar el paquete y cómo ejecutar el ejemplo en Flutter.
4. Enlace al repositorio de GitHub donde se encuentra publicado el paquete.

Evaluación

Tu prueba será evaluada en base a los siguientes criterios:

- Funcionalidad del paquete: El paquete debe ser capaz de interactuar correctamente con la Fake Store API y procesar los datos obtenidos.
- Diseño del paquete: El paquete debe estar bien estructurado y seguir las mejores prácticas de diseño de paquetes para Flutter.
- Funcionalidad del ejemplo en Flutter: El ejemplo debe demostrar de manera clara y efectiva cómo utilizar el paquete en una aplicación Flutter.
- Documentación: La documentación debe ser clara, completa y proporcionar instrucciones detalladas sobre cómo usar el paquete y ejecutar el ejemplo.

Instrucciones

1. Crea el paquete para Flutter que utilice Dart para interactuar con la Fake Store API.
2. Desarrolla un ejemplo en Flutter que demuestre el uso del paquete.
3. Publica el paquete en GitHub y asegúrate de incluir una documentación detallada.
4. Una vez completada la prueba, envía el enlace al repositorio de GitHub.

2.4 FASE 4 - Crear Un Sistema de Diseño.

Objetivo

El objetivo de esta prueba técnica es evaluar tus habilidades para crear un paquete para Flutter que contenga un sistema de diseño basado en Atomic Design. Además, se espera que incluyas un ejemplo tipo showcase que muestre cada componente del sistema de diseño funcionando.

Requerimientos

1. Creación del Paquete para Flutter:
    - Crea un paquete de Flutter que contenga un sistema de diseño basado en Atomic Design.
    - Divide el sistema de diseño en átomos, moléculas, organismos, plantillas y páginas, siguiendo los principios de Atomic Design.
    - Cada componente debe estar bien estructurado y fácilmente reutilizable.
2. Example tipo Showcase:
    - Incluye un ejemplo en Flutter que sirva como showcase del sistema de diseño.
    - El ejemplo debe mostrar cada componente del sistema de diseño en funcionamiento.
    - Debe haber una página o pantalla dedicada a cada nivel de Atomic Design (átomos, moléculas, organismos, plantillas y páginas).
3. Publicación en GitHub:
    - Publica el paquete para Flutter en GitHub.
    - Asegúrate de incluir instrucciones claras sobre cómo usar el paquete y cómo ejecutar el ejemplo en Flutter.

Entregables

1. Código fuente del paquete para Flutter que contiene el sistema de diseño basado en Atomic Design.
2. Código fuente del ejemplo tipo showcase que muestra cada componente del sistema de diseño funcionando.
3. Documentación detallada que explique cómo usar el paquete y cómo ejecutar el ejemplo en Flutter.
4. Enlace al repositorio de GitHub donde se encuentra publicado el paquete.

Evaluación

Tu prueba será evaluada en base a los siguientes criterios:

- Organización del sistema de diseño: El sistema de diseño debe estar bien estructurado y seguir los principios de Atomic Design.
- Reutilizabilidad de los componentes: Cada componente debe ser fácilmente reutilizable en diferentes partes de la aplicación.
- Funcionalidad del ejemplo tipo showcase: El ejemplo debe demostrar de manera clara y efectiva cómo utilizar cada componente del sistema de diseño.
- Documentación: La documentación debe ser clara, completa y proporcionar instrucciones detalladas sobre cómo usar el paquete y ejecutar el ejemplo.

Instrucciones

1. Crea el paquete para Flutter que contenga el sistema de diseño basado en Atomic Design.
2. Desarrolla un ejemplo tipo showcase que muestre cada componente del sistema de diseño funcionando.
3. Publica el paquete en GitHub y asegúrate de incluir una documentación detallada.
4. Una vez completada la prueba, envía el enlace al repositorio de GitHub.

2.5 FASE 5 - Crear Aplicativo E-Commerce.

Objetivo

El objetivo de esta prueba técnica es evaluar tus habilidades para desarrollar un aplicativo complejo en Flutter, específicamente una tienda eCommerce. Se espera que utilices un paquete con un sistema de diseño y otro paquete en Dart que consuma la API Fake Store para obtener datos ficticios de productos, ambos creados por ti.

Requerimientos

1. Diagrama de flujo descriptivo:
    - Diseña y estructura un diagrama de flujo que describa el funcionamiento esperado de la aplicación eCommerce.
2. Diseño y Estructura de la Aplicación:
    - Diseña y estructura la aplicación de la tienda eCommerce siguiendo las mejores prácticas de Flutter.
    - Utiliza el paquete con el sistema de diseño creado por ti para mantener consistencia visual en toda la aplicación.
3. Consumo de la API Fake Store:
    - Utiliza el paquete en Dart que consuma la API Fake Store anteriormente creada por ti para obtener datos ficticios de productos.
    - Implementa funcionalidades como la visualización de categorías de productos, la lista de productos, detalles de productos, etc.
4. Funcionalidades de la Tienda:
    - Implementa las siguientes funcionalidades:
        - Página Principal: Muestra una vista general de la tienda, promociones, productos destacados, etc.
        - Catálogo de Productos: Permite a los usuarios navegar y filtrar productos por categorías.
        - Página de Búsqueda: Permite a los usuarios buscar productos por nombre o descripción.
        - Detalle del Producto: Muestra información detallada de un producto seleccionado.

- Creación e Inicio de Sesión: Permite a los usuarios crear una cuenta o iniciar sesión.
- Página de Carrito de Compras: Muestra los productos agregados al carrito y permite a los usuarios gestionar su carrito.
- Página de Soporte y Contacto: Proporciona información de contacto y soporte para los usuarios.

5. Diseño Responsivo:
    - Haz que la aplicación sea responsive y adaptable para que se vea bien en diferentes tamaños de pantalla y orientaciones de dispositivo.

Entregables

1. Código fuente completo de la aplicación en Flutter.
2. Documentación detallada que explique la estructura de la aplicación, las funcionalidades implementadas y cualquier decisión de diseño importante.

Evaluación

Tu prueba será evaluada en base a los siguientes criterios:

- Funcionalidad: La aplicación debe cumplir con todas las funcionalidades requeridas y ofrecer una experiencia de usuario fluida.
- Diseño y Estructura: El diseño de la aplicación debe ser atractivo y coherente, y la estructura del código debe ser clara y organizada.
- Integración de paquetes: Debes integrar correctamente el paquete con el sistema de diseño y el paquete de la API Fake Store en tu aplicación.
- Responsive: La aplicación debe verse bien y funcionar correctamente en diferentes dispositivos y tamaños de pantalla.

Instrucciones

1. Desarrolla la aplicación de la tienda eCommerce en Flutter siguiendo los requerimientos mencionados.
2. Documenta la estructura de la aplicación, las funcionalidades implementadas y cualquier decisión de diseño o implementación que consideres importante.
3. Una vez completada la prueba, envía el código fuente, la documentación y las capturas de pantalla.

2.6 FASE 6 - Generar Test y Validar Cobertura.

Objetivo

El objetivo de esta prueba técnica es evaluar tus habilidades para generar tests en una aplicación de Flutter y en los paquetes creados durante tu ruta de crecimiento. Se espera que incluya tests unitarios, tests de widgets y tests de integración, y que la cobertura de los tests sea mayor al 80%.

Requerimientos

1. Generación de Tests en la Aplicación de Flutter:
    - Genera tests unitarios para las clases y funciones principales de la aplicación de Flutter.
    - Crea tests de widgets para verificar el comportamiento y la apariencia de los widgets en la interfaz de usuario.
    - Implementa tests de integración para probar la interacción entre diferentes partes de la aplicación.
2. Generación de Tests en los Paquetes Creados:
    - Genera tests unitarios para las clases y funciones principales de los paquetes creados durante tu ruta de crecimiento.
    - Crea tests de widgets para verificar el comportamiento visual de los widgets proporcionados por los paquetes.
    - Implementa tests de integración para asegurar que los paquetes funcionen correctamente en conjunto con la aplicación de Flutter.
3. Validación de Cobertura de Tests con lcov:
    - Utiliza lcov para generar un informe de cobertura de tests para la aplicación de Flutter y los paquetes creados.
    - Verifica que la cobertura de los tests sea mayor al 80% para garantizar una adecuada cobertura del código.

Recursos

1. Implementación de test en Flutter: VER LISTA VIDEOS

Entregables

1. Código fuente de los tests generados para la aplicación de Flutter y los paquetes creados.
2. Informe de cobertura de tests generado con lcov para la aplicación de Flutter y los paquetes.
3. Documentación detallada que explique los tests generados, cómo se implementaron y cualquier decisión importante tomada durante el proceso.

Evaluación

Tu prueba será evaluada en base a los siguientes criterios:

- Cobertura de Tests: La cobertura de los tests debe ser mayor al 80% tanto para la aplicación de Flutter como para los paquetes creados.
- Calidad de los Tests: Los tests deben ser efectivos para detectar errores y garantizar la calidad del código.
- Organización y Estructura: Los tests deben estar bien organizados y estructurados de acuerdo a las mejores prácticas de testing en Flutter.
- Documentación: La documentación debe ser clara y detallada, explicando cómo se implementaron los tests y cualquier consideración importante.

Instrucciones

1. Genera tests unitarios, tests de widgets y tests de integración para la aplicación de Flutter y los paquetes creados.
2. Utiliza lcov para generar un informe de cobertura de tests y asegúrate de que la cobertura sea mayor al 80%.
3. Documenta los tests generados y el proceso de generación de la cobertura.
4. Una vez completada la prueba, envía el código fuente, el informe de cobertura y la documentación.

2.7 FASE 7 - Parametrización de la app con JSON.

Objetivo

El objetivo de esta prueba técnica es evaluar tus habilidades para parametrizar una aplicación de Flutter, permitiendo la configuración de textos, imágenes, etc., a través de un archivo JSON que se leerá como asset en la aplicación.

Requerimientos

1. Creación del Archivo JSON:
    - Crea un archivo JSON que contenga los datos para parametrizar la aplicación, como textos, imágenes, colores, etc.
    - Estructura el archivo JSON de manera adecuada para que sea fácil de leer y mantener.
2. Lectura del Archivo JSON en la Aplicación de Flutter:
    - Implementa la lectura del archivo JSON como asset en la aplicación de Flutter.
    - Utiliza la información del archivo JSON para parametrizar textos, imágenes, colores, etc., en la aplicación.

3. Uso de los Datos Parametrizados:
    - Utiliza los datos parametrizados en diferentes partes de la aplicación, como en la interfaz de usuario, en widgets, en estilos, etc.
    - Asegúrate de que la aplicación utilice los datos del archivo JSON de manera coherente y efectiva.

Entregables

1. Archivo JSON con los datos de parametrización.
2. Código fuente de la aplicación de Flutter con la implementación de la lectura del archivo JSON y el uso de los datos parametrizados.
3. Documentación detallada que explique cómo se creó el archivo JSON, cómo se implementó la lectura en la aplicación de Flutter y cómo se utilizan los datos parametrizados.

Evaluación

Tu prueba será evaluada en base a los siguientes criterios:

- Correctitud de los Datos Parametrizados: Los datos del archivo JSON deben ser correctos y estar estructurados de manera adecuada.
- Implementación en la Aplicación de Flutter: La aplicación debe leer correctamente el archivo JSON como asset y utilizar los datos parametrizados de manera efectiva.
- Organización del Código: El código de la aplicación debe estar bien organizado y seguir las mejores prácticas de Flutter.
- Coherencia y Eficiencia en el Uso de los Datos: La aplicación debe utilizar los datos parametrizados de manera coherente y eficiente en toda la interfaz de usuario.

Instrucciones

1. Crea un archivo JSON con los datos de parametrización para la aplicación.
2. Implementa la lectura del archivo JSON como asset en la aplicación de Flutter.
3. Utiliza los datos parametrizados en diferentes partes de la aplicación.
4. Documenta el proceso de creación del archivo JSON, la implementación en la aplicación de Flutter y el uso de los datos parametrizados.

2.8 FASE 8 - Accesibilidad.

Objetivo

El objetivo de esta prueba técnica es evaluar tus habilidades para aplicar accesibilidad a una aplicación de Flutter utilizando Semantics. Se espera que mejores la accesibilidad de la aplicación para usuarios con discapacidades visuales o motoras.

Requerimientos

1. Identificación de Elementos Accesibles:
    - Identifica los elementos de la aplicación que deben ser accesibles para usuarios con discapacidades.
    - Prioriza los elementos clave, como botones, campos de texto, imágenes importantes, etc.
2. Implementación de Semantics:
    - Utiliza la clase Semantics de Flutter para añadir atributos semánticos a los elementos identificados.
    - Asegúrate de proporcionar etiquetas, descripciones, roles y acciones adecuadas a los elementos para mejorar su accesibilidad.
3. Pruebas de Accesibilidad:
    - Realiza pruebas de accesibilidad en la aplicación utilizando herramientas como el lector de pantalla TalkBack (para Android) o VoiceOver (para iOS).
    - Verifica que los elementos accesibles sean anunciados correctamente y que la navegación sea fluida para usuarios con discapacidades.

Recursos

1. Implementación accesibilidad en Flutter:
    - Principios de accesibilidad: VER VIDEO
    - Accesibilidad avanzada: VER VIDEO

Entregables

1. Código fuente de la aplicación de Flutter con la implementación de Semantics para mejorar la accesibilidad.
2. Documentación detallada que explique qué elementos se hicieron accesibles, cómo se implementan utilizando Semantics y los resultados de las pruebas de accesibilidad.

Evaluación

Tu prueba será evaluada en base a los siguientes criterios:

- Identificación de Elementos Accesibles: Los elementos clave de la aplicación deben ser identificados correctamente para mejorar la accesibilidad.
- Implementación de Semantics: La implementación de Semantics en la aplicación debe ser correcta y coherente con las necesidades de accesibilidad.
- Efectividad en las Pruebas de Accesibilidad: Las pruebas de accesibilidad deben demostrar que la aplicación es usable para usuarios con discapacidades visuales o motoras.
- Documentación: La documentación debe ser clara y proporcionar detalles sobre los elementos accesibles y cómo se mejoró la accesibilidad de la aplicación.

Instrucciones

1. Identifica los elementos de la aplicación que necesitan mejoras en la accesibilidad.
2. Implementa Semantics en la aplicación para mejorar la accesibilidad de los elementos identificados.
3. Realiza pruebas de accesibilidad en la aplicación y documenta los resultados.

3 ACLARACIONES

Durante el desarrollo de esta prueba técnica, es importante tener en cuenta varios principios y prácticas que contribuyen a la calidad del código y la arquitectura del software. A continuación, se detallan brevemente los aspectos clave que se deben considerar:

1. Principios SOLID:
    - Los principios SOLID, acrónimo de Single Responsibility Principle (SRP), Open/Closed Principle (OCP), Liskov Substitution Principle (LSP), Interface Segregation Principle (ISP) y Dependency Inversion Principle (DIP), son fundamentales para escribir código mantenible y extensible.
    - Cada clase y método debe tener una única responsabilidad.
    - Las clases deben estar abiertas para extensión pero cerradas para modificación.
    - Debe ser posible sustituir una clase base por una clase derivada sin cambiar el comportamiento del programa.
    - Las interfaces de una clase no deben ser forzadas a implementar métodos que no necesitan.
    - Las dependencias deben ser invertidas para reducir el acoplamiento.
2. Clean Code:
    - El código debe ser claro, legible y comprensible para cualquier desarrollador que lo lea.
    - Debe seguir convenciones de nomenclatura consistentes y significativas.
    - Deben evitarse los comentarios innecesarios explicando lo que hace el código, ya que el código en sí mismo debe ser autoexplicativo.
    - Se deben seguir las mejores prácticas de estructuración del código y de división en funciones y clases.
3. Clean Architecture:
    - La Clean Architecture propone una estructura de código que separa claramente las preocupaciones y reduce el acoplamiento entre capas.
    - La arquitectura debe estar compuesta por capas con una dirección única de dependencia, desde las capas más internas hasta las capas más externas.
    - Las capas de la aplicación deben estar bien definidas y tener responsabilidades específicas, como la lógica de negocio, la interfaz de usuario y la infraestructura.
    - Recursos utiles: Video Clean Architecture en Flutter, Artículo Clean Architecture en Flutter

4. Patrones de Diseño:
    - Los patrones de diseño son soluciones probadas para problemas comunes en el diseño de software.
    - Deben utilizarse los patrones de diseño apropiados según el contexto y los requisitos del problema a resolver.
    - Algunos ejemplos de patrones de diseño incluyen el patrón Singleton, el patrón Factory, el patrón Builder, el patrón Observer, entre otros.
    - Recursos: Lista Videos Patrones de Diseño Flutter

Durante el desarrollo de la prueba técnica, se espera que apliques estos principios y prácticas para garantizar la calidad del código y la arquitectura de la aplicación. Además, la aplicación debe estar bien estructurada, fácil de entender y modificar, y debe cumplir con los requisitos funcionales y no funcionales especificados en la prueba.

ANEXO 1

Querido desarrollador.

Entendemos que el proceso de aprendizaje puede presentar desafíos y que algunos de ustedes pueden encontrar dificultades durante la realización de la primera prueba técnica. En caso de no lograr un desempeño satisfactorio en esta prueba, queremos ofrecerles una guía para reforzar sus conocimientos en los fundamentos de Dart y Flutter.

Si se encuentran en esta situación, les recomendamos proceder a estudiar los fundamentos de Dart y Flutter desde cero. Pueden hacer uso de recursos disponibles como los tutoriales de Somos Aleteo, los cuales ofrecen una excelente introducción a Dart y Flutter con explicaciones claras y ejemplos prácticos.

Además, les recordamos que el Chapter Mobile dispone de una ruta de habilitación diseñada específicamente para reforzar los conocimientos en Dart y Flutter. Esta ruta incluye materiales de estudio, ejercicios prácticos y acompañamiento de mentores que estarán disponibles para ayudarles en su proceso de aprendizaje.

No se desanimen si encuentran dificultades en esta primera prueba. Aprovechen esta oportunidad para fortalecer sus bases en Dart y Flutter, ya que estos conocimientos serán fundamentales para su desarrollo como desarrolladores móviles.

Estamos aquí para apoyarlos en su camino de aprendizaje y crecimiento. No duden en acudir a nosotros si necesitan orientación o ayuda adicional.

¡Mucho ánimo y éxito en sus estudios!

Recursos adicionales para el aprendizaje:

1. Videos - Hacia la maestría en ciencias de computación con Dart y Flutter
   - LISTA DE VIDEOS

2. Video - POO - Programación orientada a objetos
   - VER VIDEO

3. Artículo - Ciclo de vida de las pantallas
   - VER ARTÍCULO

4. Artículo - Linters en VS Code
   - VER ARTÍCULO
   - DOCUMENTACIÓN DART

5. Videos - Canal de Youtube SOMOSALETEO
   - VER CANAL

6. Artículo - Profiling y rendimiento
   - VER ARTICULO

4 GENERALES

3.1 Información de la versión 0.2

Elaborado por	Dolph Hincapié	**Fecha**	2024/05/06
Revisado por	Albert Joao Jiménez Peralta	**Fecha**	2024/05/07